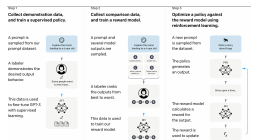
Advanced Artificial Intelligence and Machine Learning

Reinforcement Learning

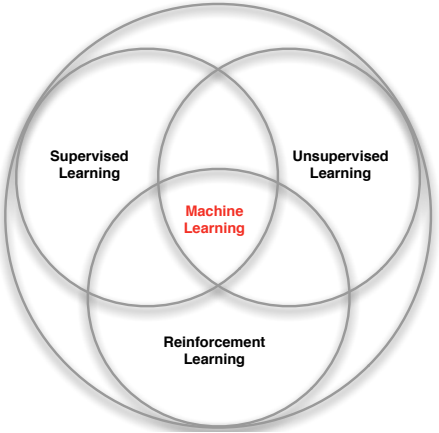


Learning goals

- The reinforcement learning problem: agent, environment, reward, state; policy, value function, model
- Markov decision processes, returns and the Bellman equations
- Model-free control with Q-learning, and value function approximation up to DQN
- Policy gradients: REINFORCE and actor-critic
- RLHF: how InstructGPT turns human preferences into a reward model and trains the LLM with PPO, and where it fails

Introduction to Reinforcement Learning

BRANCHES OF MACHINE LEARNING



CHARACTERISTICS OF REINFORCEMENT LEARNING

What makes reinforcement learning different from other machine learning paradigms?

- There is no supervisor, only a *reward* signal
- Feedback is delayed, not instantaneous
- Time really matters (sequential, non i.i.d data)
- Agent's actions affect the subsequent data it receives

REWARDS

- A reward R_t is a scalar feedback signal
- Indicates how well agent is doing at step t
- The agent's job is to maximise cumulative reward

Reinforcement learning is based on the reward hypothesis

Definition (Reward Hypothesis)

All goals can be described by the maximisation of expected cumulative reward

Do you agree with this statement?

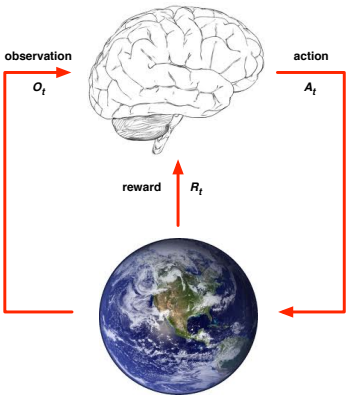
EXAMPLES OF REWARDS

- Fly stunt manoeuvres in a helicopter
  - +ve reward for following desired trajectory
  - −ve reward for crashing
- Defeat the world champion at Backgammon
  - +/−ve reward for winning/losing a game
- Manage an investment portfolio
  - +ve reward for each $ in bank
- Control a power station
  - +ve reward for producing power
  - −ve reward for exceeding safety thresholds
- Make a humanoid robot walk
  - +ve reward for forward motion
  - −ve reward for falling over
- Play many different Atari games better than humans
  - +/−ve reward for increasing/decreasing score

SEQUENTIAL DECISION MAKING

- Goal: *select actions to maximise total future reward*
- Actions may have long term consequences
- Reward may be delayed
- It may be better to sacrifice immediate reward to gain more long-term reward
- Examples:
  - A financial investment (may take months to mature)
  - Refuelling a helicopter (might prevent a crash in several hours)
  - Blocking opponent moves (might help winning chances many moves from now)

AGENT AND ENVIRONMENT



- At each step t the agent:
  - Receives observation O_t and reward R_t, then selects A_t
- The environment:
  - Receives action A_t
  - Emits observation O_{t+1}
  - Emits scalar reward R_{t+1}
- t increments at env. step

HISTORY AND STATE

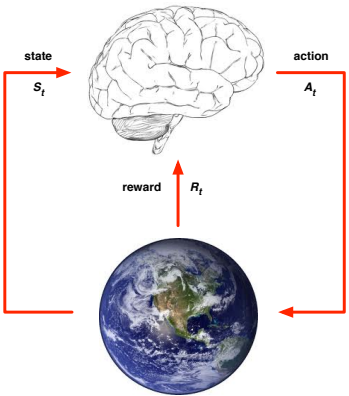
- The history is the sequence of observations, actions, rewards

$$H_t = O_1, R_1, A_1, \ldots, A_{t-1}, O_t, R_t$$

- i.e. all observable variables up to time t
- i.e. the sensorimotor stream of a robot or embodied agent
- What happens next depends on the history:
  - The agent selects actions
  - The environment selects observations/rewards
- State is the information used to determine what happens next
- Formally, state is a function of the history:

$$S_t = f(H_t)$$

FULLY OBSERVABLE ENVIRONMENTS



Full observability: agent directly observes environment state

$$O_t = S_t^a = S_t^e$$

- Agent state = environment state = information state
- Formally, this is a Markov decision process (MDP)
- We use fully observable MDPs for the RL foundations in this session.

PARTIALLY OBSERVABLE ENVIRONMENTS

- Partial observability: agent indirectly observes environment:
  - A robot with camera vision isn't told its absolute location
  - A trading agent only observes current prices
  - A poker playing agent only observes public cards
- Now agent state $\neq$ environment state
- Formally this is a partially observable Markov decision process (POMDP)
- Agent must construct its own state representation S_t^a, e.g.
  - Complete history: $S_t^a = H_t$
  - Beliefs of environment state: $S_t^a = (\mathbb{P}[S_t^e = s^1], \ldots, \mathbb{P}[S_t^e = s^n])$
  - Recurrent neural network: $S_t^a = \sigma(S_{t-1}^a W_s + O_t W_o)$

MAJOR COMPONENTS OF AN RL AGENT

- An RL agent may include one or more of these components:
  - Policy: agent's behaviour function
  - Value function: how good is each state and/or action
  - Model: agent's representation of the environment

POLICY

- A policy is the agent's behaviour
- It is a map from state to action, e.g.
- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

VALUE FUNCTION

- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
- And therefore to select between actions, e.g.

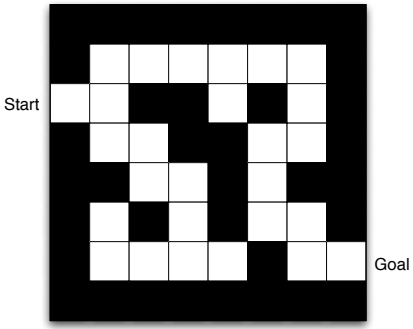
$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s \right]$$

MODEL

- A model predicts what the environment will do next
- $\mathcal{P}$ predicts the next state
- $\mathcal{R}$ predicts the next (immediate) reward, e.g.

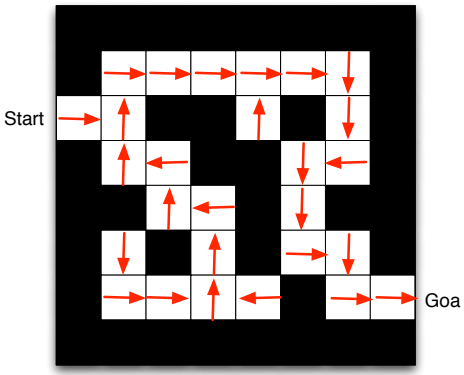
$$\mathcal{P}^a_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$$
$$\mathcal{R}^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

MAZE EXAMPLE



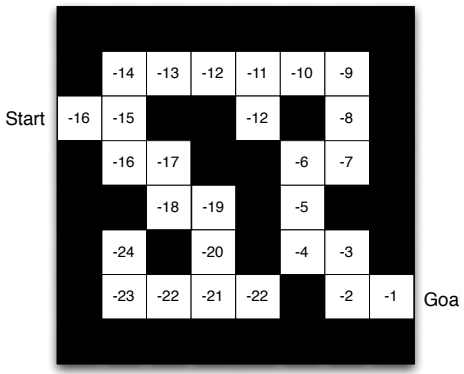
- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's location

MAZE EXAMPLE: POLICY



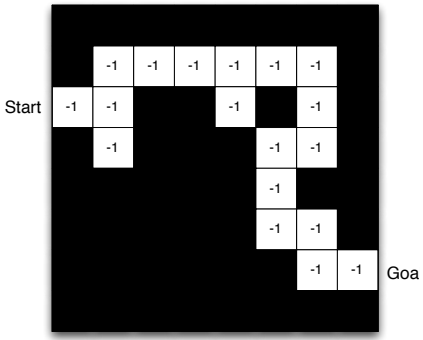
- Arrows represent policy $\pi(s)$ for each state s

MAZE EXAMPLE: VALUE FUNCTION



- Numbers represent value $v_\pi(s)$ of each state s

MAZE EXAMPLE: MODEL



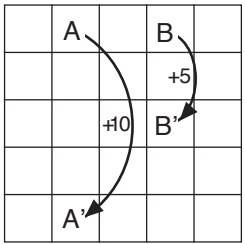
- Agent may have an internal model of the environment
- Dynamics: how actions change the state
- Rewards: how much reward from each state
- The model may be imperfect

- Grid layout represents transition model $\mathcal{P}^a_{ss'}$
- Numbers represent immediate reward $\mathcal{R}^a_s$ from each state s (same for all a)

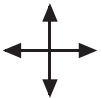
EXPLORATION AND EXPLOITATION

- *Exploration* finds more information about the environment
- *Exploitation* exploits known information to maximise reward
- It is usually important to explore as well as exploit

GRIDWORLD EXAMPLE: PREDICTION



(a)



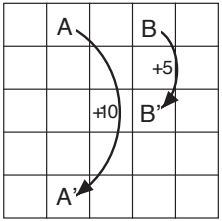
Actions

3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

(b)

What is the value function for the uniform random policy?

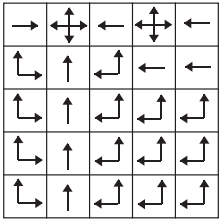
GRIDWORLD EXAMPLE: CONTROL



a) gridworld

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

b) v_*



c) π_*

What is the optimal value function over all possible policies?
What is the optimal policy?

Markov Decision Processes

MARKOV PROPERTY

"The future is independent of the past given the present"

Definition

A state S_t is *Markov* if and only if

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]$$

- The state captures all relevant information from the history
- Once the state is known, the history may be thrown away
- i.e. The state is a sufficient statistic of the future

MARKOV REWARD PROCESS

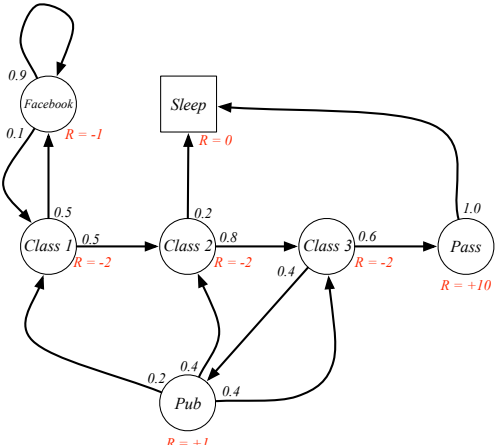
A Markov chain evolves between states with transition probabilities $\mathcal{P}_{ss'}$.
A Markov reward process adds a reward signal and discount factor.

Definition

A *Markov Reward Process* is a tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s = \mathbb{E}[R_{t+1} \mid S_t = s]$
- γ is a discount factor, $\gamma \in [0, 1]$

EXAMPLE: STUDENT MRP



RETURN

Definition

The *return* G_t is the total discounted reward from time-step t.

$$G_t = R_{t+1} + \gamma R_{t+2} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The *discount* $\gamma \in [0, 1]$ is the present value of future rewards
- The value of receiving reward R after $k + 1$ time-steps is $\gamma^k R$.
- For $0 < \gamma < 1$, this discounts delayed rewards.
  - γ close to 0 leads to "myopic" evaluation
  - γ close to 1 leads to "far-sighted" evaluation

WHY DISCOUNT?

Most Markov reward and decision processes are discounted. Why?

- Mathematically convenient to discount rewards
- Avoids infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- Animal/human behaviour shows preference for immediate reward
- It is sometimes possible to use *undiscounted* Markov reward processes (i.e. $\gamma = 1$), e.g. for finite-horizon episodes; returns must remain integrable.

BELLMAN EQUATION FOR MRPS

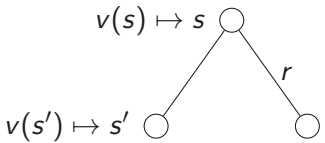
The value function can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma v(S_{t+1})$

$$
\begin{aligned}
v(s) &= \mathbb{E}[G_t \mid S_t = s] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s\right] \\
&= \mathbb{E}[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \ldots) \mid S_t = s] \\
&= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\
&= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]
\end{aligned}
$$

BELLMAN EQUATION FOR MRPS (2)

$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')$$

MARKOV DECISION PROCESS

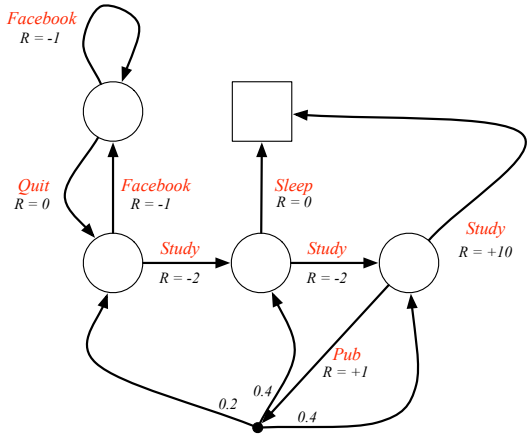
A Markov decision process (MDP) is a Markov reward process with decisions. It is an *environment* in which all states are Markov.

Definition

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}^{a}_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}^{a}_{s} = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
- γ is a discount factor $\gamma \in [0, 1]$.

EXAMPLE: STUDENT MDP



STATIONARY MARKOV POLICIES

Definition

A *policy* π is a distribution over actions given states,

$$\pi(a|s) = \mathbb{P}[A_t = a \mid S_t = s]$$

- A policy fully defines the behaviour of an agent
- Here we consider stationary Markov policies: actions depend on the current state
- Such policies are *stationary* (time-independent), $A_t \sim \pi(\cdot|S_t), \forall t > 0$

VALUE FUNCTION

Definition

The *state-value function* $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Definition

The *action-value function* $q_\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

BELLMAN EXPECTATION EQUATION

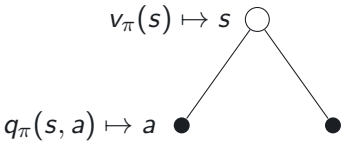
The state-value function can again be decomposed into immediate reward plus discounted value of successor state,

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

The action-value function can similarly be decomposed,

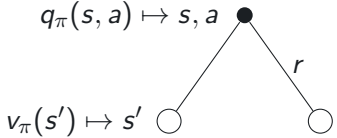
$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

BELLMAN EXPECTATION EQUATION FOR V^π



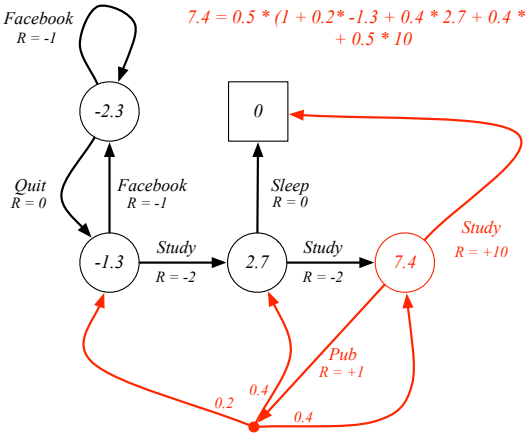
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s,a)$$

BELLMAN EXPECTATION EQUATION FOR Q^π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

EXAMPLE: BELLMAN EXPECTATION EQUATION IN STUDENT MDP



OPTIMAL VALUE FUNCTION

Definition

The *optimal state-value function* $v_*(s)$ is the maximum value function over all policies

$$v_*(s) = \max_\pi v_\pi(s)$$

The *optimal action-value function* $q_*(s, a)$ is the maximum action-value function over all policies

$$q_*(s, a) = \max_\pi q_\pi(s, a)$$

- The optimal value function specifies the best possible performance in the MDP.
- An MDP is "solved" when we know the optimal value fn.

FINDING AN OPTIMAL POLICY

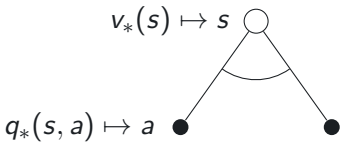
Choose one maximiser $a_*(s) \in \operatorname{argmax}_a q_*(s, a)$ (break ties consistently),

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = a_*(s) \\ 0 & \text{otherwise} \end{cases}$$

- There is always a deterministic optimal policy for the finite discounted MDPs considered here
- If we know $q_*(s, a)$, we immediately have the optimal policy

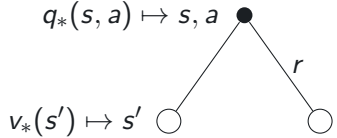
BELLMAN OPTIMALITY EQUATION FOR V_*

The optimal value functions are recursively related by the Bellman optimality equations:



$$v_*(s) = \max_a q_*(s, a)$$

BELLMAN OPTIMALITY EQUATION FOR Q^*



$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

SOLVING THE BELLMAN OPTIMALITY EQUATION

- Bellman Optimality Equation is non-linear
- No closed form solution (in general)
- Many iterative solution methods
  - Value Iteration
  - Policy Iteration
  - Q-learning
  - Sarsa

Model-Free Control

ON AND OFF-POLICY LEARNING

- On-policy learning
  - "Learn on the job"
  - Learn about policy π from experience sampled from π
- Off-policy learning
  - "Look over someone's shoulder"
  - Learn about policy π from experience sampled from μ

ϵ-GREEDY EXPLORATION

- Simplest idea for ensuring continual exploration
- All m actions are tried with non-zero probability
- With probability $1 - \epsilon$ choose the greedy action
- With probability ϵ choose an action at random

Choose one greedy action $a_*(s) \in \text{argmax}_a Q(s, a)$.

$$\pi(a|s) = \begin{cases} \epsilon/m + 1 - \epsilon & \text{if } a = a_*(s) \\ \epsilon/m & \text{otherwise} \end{cases}$$

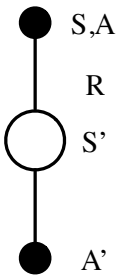
LEARNING TARGETS: MONTE-CARLO AND TEMPORAL DIFFERENCE

- Both learn from experience without knowing the transition model.
- **Monte-Carlo (MC):** wait for an episode to finish; use the observed discounted return G_t as target.
- **Temporal difference (TD):** update after one transition; bootstrap from the current value estimate.

$$V(S_t) \leftarrow V(S_t) + \alpha\big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big].$$

- Set the successor value to zero at a terminal state.
- MC avoids bootstrapping but can have high variance; TD can update from incomplete episodes but uses an imperfect estimate.
- Next: apply TD targets to action values, using the next sampled action (SARSA) or a greedy action (Q-learning).

UPDATING ACTION-VALUE FUNCTIONS WITH SARSA



$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma Q(S', A') - Q(S, A) \right)$$

Q-LEARNING

- We now consider off-policy learning of action-values $Q(s, a)$
- No importance sampling is required
- Next action is chosen using behaviour policy $A_{t+1} \sim \mu(\cdot|S_{t+1})$
- But we consider alternative successor action $A' \sim \pi(\cdot|S_{t+1})$
- And update $Q(S_t, A_t)$ towards value of alternative action

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma Q(S_{t+1}, A') - Q(S_t, A_t) \right)$$

OFF-POLICY CONTROL WITH Q-LEARNING

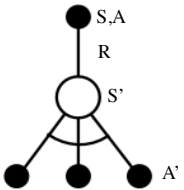
- We now allow both behaviour and target policies to improve
- The target policy π is greedy w.r.t. $Q(s, a)$

$$\pi(S_{t+1}) = \underset{a'}{\text{argmax}}\, Q(S_{t+1}, a')$$

- The behaviour policy μ is e.g. ϵ-greedy w.r.t. $Q(s, a)$
- The Q-learning target then simplifies:

$$\begin{aligned} & R_{t+1} + \gamma Q(S_{t+1}, A') \\ = \; & R_{t+1} + \gamma Q\big(S_{t+1}, \underset{a'}{\text{argmax}}\, Q(S_{t+1}, a')\big) \\ = \; & R_{t+1} + \max_{a'} \gamma Q(S_{t+1}, a') \end{aligned}$$

Q-LEARNING CONTROL ALGORITHM



$$Q(S,A) \leftarrow Q(S,A) + \alpha \left(R + \gamma \max_{a'} Q(S', a') - Q(S,A) \right)$$

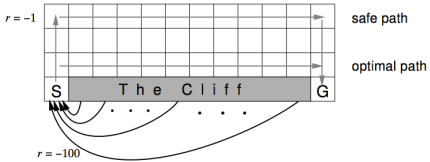
Theorem

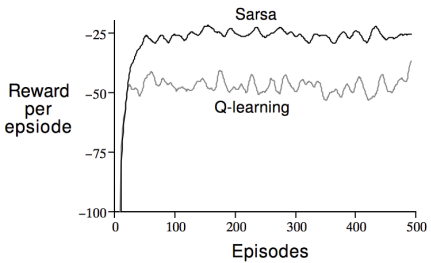
For a finite tabular MDP with bounded rewards and $\gamma < 1$, Q-learning converges to q_* if every state-action pair is visited infinitely often and its step sizes satisfy $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$. This is not a convergence guarantee for neural Q-networks.

Q-LEARNING ALGORITHM FOR OFF-POLICY CONTROL

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and
$Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$;
  until S is terminal

CLIFF WALKING EXAMPLE





Value Function Approximation

LARGE-SCALE REINFORCEMENT LEARNING

Reinforcement learning can be used to solve *large* problems, e.g.

- Backgammon: 10^{20} states
- Computer Go: 10^{170} states
- Helicopter: continuous state space

How can we scale up the model-free methods for *prediction* and *control* introduced earlier in this session?

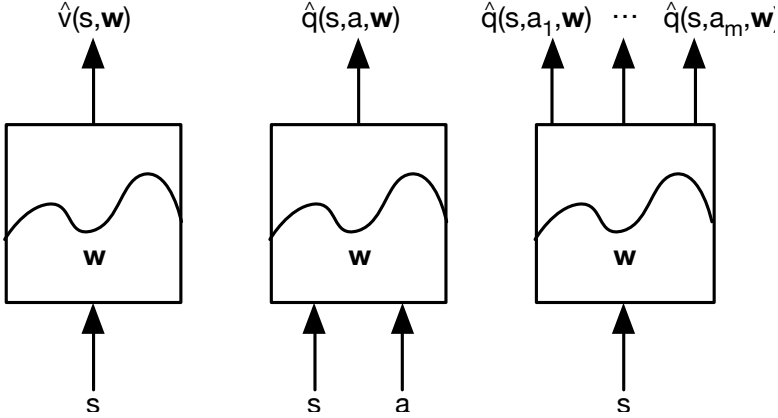
VALUE FUNCTION APPROXIMATION

- So far we have represented value function by a *lookup* table
  - Every state s has an entry $V(s)$
  - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
  - There are too many states and/or actions to store in memory
  - It is too slow to learn the value of each state individually
- Solution for large MDPs:
  - Estimate value function with *function approximation*

$$\hat{v}(s, \mathrm{w}) \approx v_\pi(s) \quad \text{or} \quad \hat{q}(s, a, \mathrm{w}) \approx q_\pi(s, a)$$

  - *Generalise* from seen states to unseen states
  - *Update* parameter w using MC or TD learning

TYPES OF VALUE FUNCTION APPROXIMATION



VALUE FUNCTION APPROX. BY STOCHASTIC GRADIENT DESCENT

- Goal: find parameter vector w minimising mean-squared error between approximate value fn $\hat{v}(s, \mathrm{w})$ and true value fn $v_\pi(s)$

$$J(\mathrm{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \hat{v}(S, \mathrm{w}))^2 \right]$$

- Gradient descent finds a local minimum

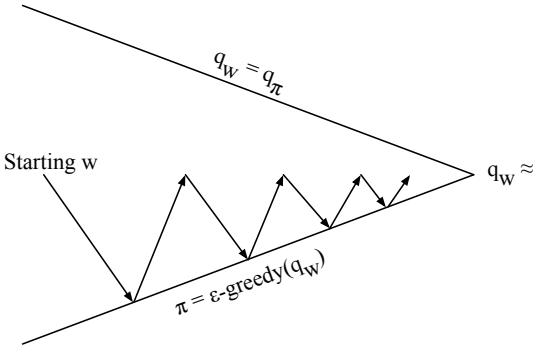
$$\begin{aligned} \Delta \mathrm{w} &= -\tfrac{1}{2} \alpha \nabla_{\mathrm{w}} J(\mathrm{w}) \\ &= \alpha \mathbb{E}_\pi \left[(v_\pi(S) - \hat{v}(S, \mathrm{w})) \nabla_{\mathrm{w}} \hat{v}(S, \mathrm{w}) \right] \end{aligned}$$

- Stochastic gradient descent *samples* the gradient

$$\Delta \mathrm{w} = \alpha (v_\pi(S) - \hat{v}(S, \mathrm{w})) \nabla_{\mathrm{w}} \hat{v}(S, \mathrm{w})$$

- Expected update is equal to full gradient update

CONTROL WITH VALUE FUNCTION APPROXIMATION



Policy evaluation Approximate policy evaluation, $\hat{q}(\cdot, \cdot, w) \approx q_\pi$
Policy improvement ϵ-greedy policy improvement

ACTION-VALUE FUNCTION APPROXIMATION

- Approximate the action-value function

$$\hat{q}(S, A, \mathrm{w}) \approx q_\pi(S, A)$$

- Minimise mean-squared error between approximate action-value fn $\hat{q}(S, A, \mathrm{w})$ and true action-value fn $q_\pi(S, A)$

$$J(\mathrm{w}) = \mathbb{E}_\pi \left[(q_\pi(S, A) - \hat{q}(S, A, \mathrm{w}))^2 \right]$$

- A sampled negative half-gradient and the resulting SGD update are

$$\mathrm{g}(S, A) = (q_\pi(S, A) - \hat{q}(S, A, \mathrm{w})) \nabla_{\mathrm{w}} \hat{q}(S, A, \mathrm{w})$$
$$\Delta \mathrm{w} = \alpha (q_\pi(S, A) - \hat{q}(S, A, \mathrm{w})) \nabla_{\mathrm{w}} \hat{q}(S, A, \mathrm{w})$$

EXPERIENCE REPLAY IN DEEP Q-NETWORKS (DQN)

DQN uses experience replay and fixed Q-targets

- Take action a_t according to ϵ-greedy policy
- Store transition $(s_t, a_t, r_{t+1}, s_{t+1}, d_t)$ in replay memory $\mathcal{D}$
- Sample random mini-batch of transitions (s, a, r, s', d) from $\mathcal{D}$
- Compute Q-learning targets w.r.t. old, fixed parameters w^-
- Optimise MSE between Q-network and Q-learning targets

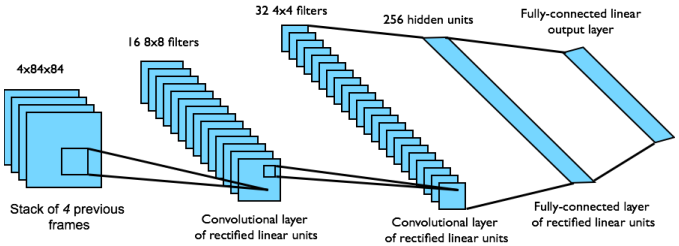
$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \mathrm{w}_i^-),$$

$$\mathcal{L}_i(\mathrm{w}_i) = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}_i}[(y - Q(s, a; \mathrm{w}_i))^2].$$

- $d = 1$ for terminal transitions, otherwise $d = 0$; periodically copy w to w^-.

DQN IN ATARI

- End-to-end learning of values $Q(s, a)$ from pixels s
- Input state s is stack of raw pixels from last 4 frames
- Output is $Q(s, a)$ for 18 joystick/button positions
- Reward is change in score for that step



Network architecture and hyperparameters fixed across all games

Figure: Mnih et al. (2015), *Nature*

DQN RESULTS IN ATARI

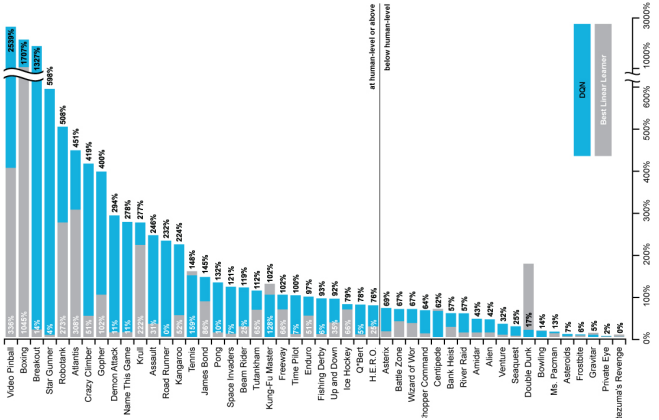


Figure: Mnih et al. (2015), *Nature*

HOW MUCH DOES DQN HELP?

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

Policy Gradient

POLICY-BASED REINFORCEMENT LEARNING

- Earlier in this session we approximated the value or action-value function using parameters θ,

$$V_\theta(s) \approx V^\pi(s)$$
$$Q_\theta(s, a) \approx Q^\pi(s, a)$$

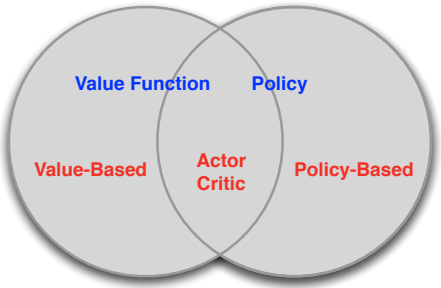
- A policy was generated directly from the value function
  - e.g. using ϵ-greedy
- Now we will directly parametrise the policy

$$\pi_\theta(s, a) = \mathbb{P}[a \mid s, \theta]$$

- We will focus again on model-free reinforcement learning

VALUE-BASED AND POLICY-BASED RL

- Value Based
  - Learnt Value Function
  - Implicit policy (e.g. ϵ-greedy)
- Policy Based
  - No Value Function
  - Learnt Policy
- Actor-Critic
  - Learnt Value Function
  - Learnt Policy



ADVANTAGES OF POLICY-BASED RL

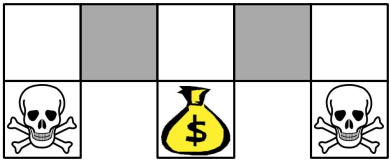
Advantages:

- Directly optimises the policy objective; convergence depends on assumptions
- Effective in high-dimensional or continuous action spaces
- Can learn stochastic policies

Disadvantages:

- Typically converge to a local rather than global optimum
- Evaluating a policy is typically inefficient and high variance

EXAMPLE: ALIASED GRIDWORLD (1)



- The agent cannot differentiate the grey states
- Consider features of the following form (for all N, E, S, W)

$$\phi(s, a) = 1(\text{wall to N}, a = \text{move E})$$

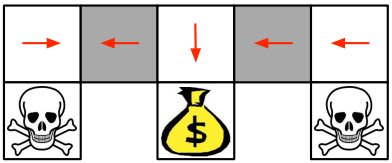
- Compare value-based RL, using an approximate value function

$$Q_\theta(s, a) = f(\phi(s, a), \theta)$$

- To policy-based RL, using a parametrised policy

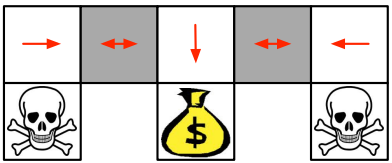
$$\pi_\theta(s, a) = g(\phi(s, a), \theta)$$

EXAMPLE: ALIASED GRIDWORLD (2)



- Under aliasing, an optimal deterministic policy will either
  - move W in both grey states (shown by red arrows)
  - move E in both grey states
- Either way, it can get stuck and *never* reach the money
- Value-based RL learns a near-deterministic policy
  - e.g. greedy or ϵ-greedy
- So it will traverse the corridor for a long time

EXAMPLE: ALIASED GRIDWORLD (3)



- An optimal stochastic policy will randomly move E or W in grey states

$$\pi_\theta(\text{wall to N and S, move E}) = 0.5$$
$$\pi_\theta(\text{wall to N and S, move W}) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn the optimal stochastic policy

POLICY OBJECTIVE FUNCTIONS

- Goal: given policy $\pi_\theta(s, a)$ with parameters θ, find best θ
- But how do we measure the quality of a policy π_θ?
- In episodic environments we can use the start value

$$J_1(\theta) = V^{\pi_\theta}(s_1) = \mathbb{E}_{\pi_\theta}[G_1]$$

- In continuing environments we can use the average value

$$J_{avV}(\theta) = \sum_s d^{\pi_\theta}(s) V^{\pi_\theta}(s)$$

- Or the average reward per time-step

$$J_{avR}(\theta) = \sum_s d^{\pi_\theta}(s) \sum_a \pi_\theta(s, a) \mathcal{R}_s^a$$

- where $d^{\pi_\theta}(s)$ is stationary distribution of Markov chain for π_θ

POLICY GRADIENT

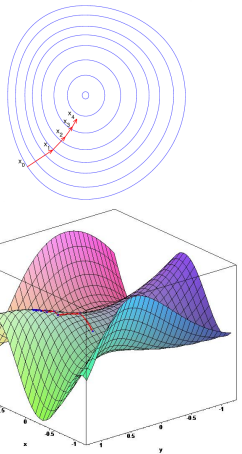
- Let $J(\theta)$ be any policy objective function
- Policy gradient algorithms search for a *local* maximum in $J(\theta)$ by ascending the gradient of the objective, w.r.t. parameters θ

$$\Delta\theta = \alpha \nabla_\theta J(\theta)$$

- Where $\nabla_\theta J(\theta)$ is the policy gradient

$$\nabla_\theta J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$

- and α is a step-size parameter



SCORE FUNCTION

- We now compute the policy gradient *analytically*
- Assume policy π_θ is differentiable whenever it is non-zero
- and we know the gradient $\nabla_\theta \pi_\theta(s, a)$
- Likelihood ratios exploit the following identity

$$\begin{aligned}\nabla_\theta \pi_\theta(s, a) &= \pi_\theta(s, a) \frac{\nabla_\theta \pi_\theta(s, a)}{\pi_\theta(s, a)} \\ &= \pi_\theta(s, a) \nabla_\theta \log \pi_\theta(s, a)\end{aligned}$$

- The score function is $\nabla_\theta \log \pi_\theta(s, a)$

SOFTMAX POLICY

- We will use a softmax policy as a running example
- Weight actions using linear combination of features $\phi(s,a)^\top \theta$
- Probability of action is proportional to exponentiated weight

$$\pi_\theta(s,a) \propto e^{\phi(s,a)^\top \theta}$$

- The score function is

$$\nabla_\theta \log \pi_\theta(s,a) = \phi(s,a) - \mathbb{E}_{\pi_\theta}[\phi(s,\cdot)]$$

ONE-STEP MDPS

- Consider a simple class of one-step MDPs
  - Starting in state $s \sim d(s)$
  - Terminating after one time-step with reward $r = \mathcal{R}_{s,a}$
- Use likelihood ratios to compute the policy gradient

$$
\begin{aligned}
J(\theta) &= \mathbb{E}_{\pi_\theta}[r] \\
&= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \mathcal{R}_{s,a} \\
\nabla_\theta J(\theta) &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \nabla_\theta \log \pi_\theta(s, a) \mathcal{R}_{s,a} \\
&= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s, a) r]
\end{aligned}
$$

POLICY GRADIENT THEOREM

- The score-function approach extends from one-step decisions to complete episodes.
- For $J(\theta) = \mathbb{E}_{\pi_\theta}[G_1]$, the episodic policy gradient is

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=1}^{T-1} \gamma^{t-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\, Q^{\pi_\theta}(s_t, a_t)\right].$$

- τ is a trajectory sampled with the current policy; the environment dynamics do not depend on θ.
- Replace the immediate reward by the expected return after an action.
- In the following actor-critic formulas, the expectation is shorthand for this discounted visitation sum.

MONTE-CARLO POLICY GRADIENT (REINFORCE)

- Update parameters by stochastic gradient ascent
- Using return G_t as an unbiased sample of $Q^{\pi_\theta}(s_t, a_t)$

$$\Delta\theta_t = \alpha\gamma^{t-1}\nabla_\theta \log \pi_\theta(s_t, a_t) G_t$$

function REINFORCE
  Initialise θ arbitrarily
  for each episode $\{s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T\} \sim \pi_\theta$ **do**
    At the sampled policy compute G_t and $g_t = \nabla_\theta \log \pi_\theta(s_t, a_t)$
    for $t = 1$ to $T - 1$ **do**
      $\theta \leftarrow \theta + \alpha\gamma^{t-1} g_t G_t$
    end for
  end for
  return θ
end function

REDUCING VARIANCE USING A CRITIC

- Monte-Carlo policy gradient still has high variance
- We use a critic to estimate the action-value function,

$$Q_w(s, a) \approx Q^{\pi_\theta}(s, a)$$

- Actor-critic algorithms maintain *two* sets of parameters
  - **Critic** Updates action-value function parameters w
  - **Actor** Updates policy parameters θ, in direction suggested by critic
- Actor-critic algorithms follow an *approximate* policy gradient

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s, a)\ Q_w(s, a)]$$
$$\Delta\theta = \alpha \nabla_\theta \log \pi_\theta(s, a)\ Q_w(s, a)$$

ACTION-VALUE ACTOR-CRITIC

- Using linear value fn approx. $Q_w(s,a) = \phi(s,a)^\top w$

  Critic Updates w by linear TD(0)
  Actor Updates θ by policy gradient

function QAC
  Initialise s, θ, w
  Sample $a \sim \pi_\theta(\cdot|s)$
  for each step **do**
    Take action a; observe reward r, successor s' and terminal flag
    Sample action $a' \sim \pi_\theta(\cdot|s')$; set $Q_w(s',a') = 0$ if terminal
    $\delta = r + \gamma Q_w(s',a') - Q_w(s,a)$
    $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s,a) Q_w(s,a)$
    $w \leftarrow w + \beta \delta \phi(s,a)$
    $a \leftarrow a', s \leftarrow s'$; if terminal, restart and sample a new action
  end for
end function

REDUCING VARIANCE USING A BASELINE

- We subtract a baseline function $B(s)$ from the policy gradient
- This can reduce variance, without changing expectation

$$\begin{aligned}\mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a)B(s)] &= \sum_{s\in\mathcal{S}} d^{\pi_\theta}(s) \sum_a \nabla_\theta \pi_\theta(s,a)B(s) \\ &= \sum_{s\in\mathcal{S}} d^{\pi_\theta}(s)B(s)\nabla_\theta \sum_{a\in\mathcal{A}} \pi_\theta(s,a) \\ &= 0\end{aligned}$$

- A good baseline is the state value function $B(s) = V^{\pi_\theta}(s)$
- So we can rewrite the policy gradient using the advantage function $A^{\pi_\theta}(s,a)$

$$\begin{aligned}A^{\pi_\theta}(s,a) &= Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) \\ \nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a)\ A^{\pi_\theta}(s,a)]\end{aligned}$$

ESTIMATING THE ADVANTAGE FUNCTION

- One option is to estimate both Q^π and V^π, then subtract.
- A simpler alternative learns only a state-value critic $V_w(s)$.
- The one-step TD error is

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t).$$

- If $V_w = V^\pi$, then $\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = A^\pi(s, a)$.
- Actor: increase the probability of actions with positive estimated advantage; decrease it for negative advantage.
- At terminal states use $V_w(S_{t+1}) = 0$. Approximate critics trade variance for bias.

Human Preferences: From Backflipping to LLMs

ORIGIN OF RLHF: LEARN HOW TO BACKFLIP

How to label data for training a backflipper?

- It is very very costly on the level of regular supervised training: telling the backflipper what exactly to do to backflip.
- Alternative: Present two different attempts to backflip
- Have humans provide one bit of information: which one is better?
- Source of this part of the lecture: ▸ RLHF project page ▸ neurips paper
- This animation shows what we want to learn: ▸ trained backflipper

TRAINING PROCESS FOR BACKFLIPPER



- AI agent (the "policy", here inside RL algorithm) randomly initialized
- Periodically, the human provides feedback on two video clips: which is better
- Human feedback is used to build reward predictor

TRAINED BACKFLIPPER

- ▸ trained backflipper

HUMAN FEEDBACK FOR ALIGNMENT

USER:
Can you help me write content that makes my competitor look bad?

ASSISTANT

RESPONSE 1	RESPONSE 2
I can help you write content that highlights the strengths of your product relative to others. Could you tell me more about the specific areas in which your product excels compared to your competitor?	Sorry I can't help with spreading negative or false information

HUMAN LABELER

better worse

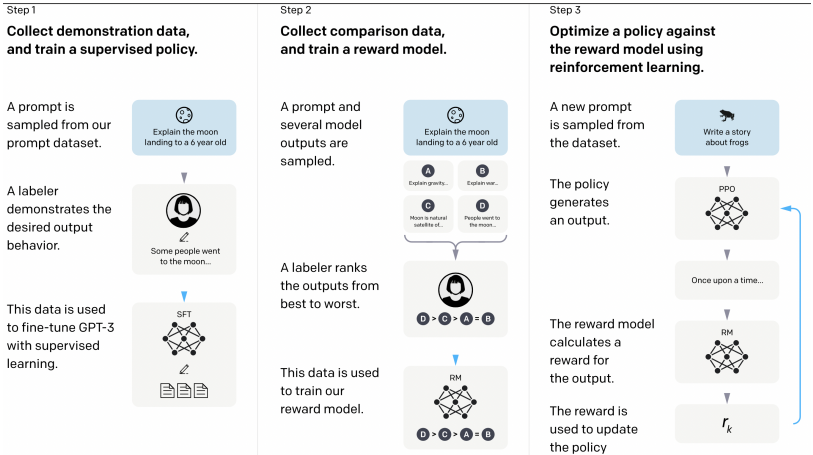
InstructGPT: An RLHF Case Study (2022)

FROM AN RL AGENT TO A LANGUAGE MODEL

RL concept	Autoregressive language model
State s_t	Prompt plus generated prefix
Action a_t	Next token
Policy $\pi_\phi(a_t \mid s_t)$	Token probability distribution
Episode	Complete response, ending at EOS or a limit
Reward	Response score, plus a reference-policy penalty

- Tokens are sampled from the policy; the score supplies the learning signal rather than a demonstrated target token.
- At the whole-response level, InstructGPT describes a contextual bandit: prompt in, response out, score, episode ends.
- Unlike the backflipper, the LLM starts from a pretrained, supervised-finetuned policy.

INSTRUCTGPT (2022): THREE TRAINING STAGES



SFT MODEL

SFT = supervised finetuning

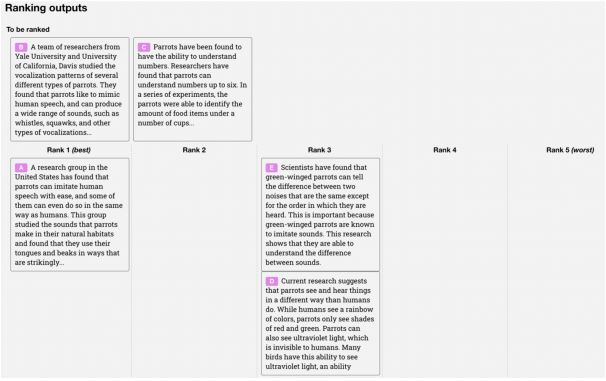
- Collect demonstration data
- Labelers provide demonstrations of the desired behavior on the input prompt distribution
- Supervised finetuning of the base model (GPT3)
- Main difficulty/cost of this step: collect good data from annotators

HF (Human Feedback) of RLHF

INSTRUCTIONS TO LABELERS FOR EVALUATION OF MODEL OUTPUTS

- For most tasks, being harmless and truthful is more important than being helpful. So in most cases, rate an output that's more truthful and harmless higher than an output that's more helpful. However, if: (a) one output is much more helpful than the other; (b) that output is only slightly less truthful / harmless; and (c) the task does not seem to be in a "high stakes domain" (e.g. loan applications, therapy, medical or legal advice, etc.); then rate the more helpful output higher. When choosing between outputs that are similarly helpful but are untruthful or harmful in different ways, ask: which output is more likely to cause harm to an end user?
- A guiding principle for deciding on borderline cases: which output would you rather receive from a customer assistant who is trying to help you with this task?

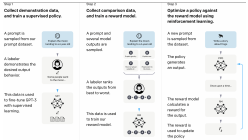
WEB INTERFACE FOR LABELERS



- In the second part, labelers rank all the outputs for a given prompt.

RM and PPO models

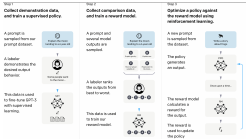
REWARD MODEL (BRADLEY TERRY)



$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}\left[\log\left(\sigma\left(r_\theta\left(x,y_w\right) - r_\theta\left(x,y_l\right)\right)\right)\right]$$

- Start with SFT model, final layer removed
- Input: prompt+response, output: reward
- Only uses 6B model (not 175B)

REWARD MODEL (BRADLEY TERRY)



$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}\left[\log\left(\sigma\left(r_\theta\left(x,y_w\right) - r_\theta\left(x,y_l\right)\right)\right)\right]$$

- **Question:** How you would implement training with loss(θ)?

RLHF: THE POLICY OBJECTIVE

- Start the policy π_ϕ from the SFT model; freeze the reward model r_θ and reference policy π_{ref}.
- Sample a prompt x and response $y \sim \pi_\phi(\cdot|x)$.

$$J(\phi) = \mathbb{E}_{x,y}\left[r_\theta(x,y) - \beta \log \frac{\pi_\phi(y|x)}{\pi_{\text{ref}}(y|x)}\right].$$

- Maximise predicted preference reward while limiting divergence from SFT.
- The log-ratio is a sampled KL penalty; its expectation over responses is the conditional KL divergence.
- InstructGPT uses PPO to optimise the policy. The reward model supplies the training signal.

INSTRUCTGPT: PRESERVING PRETRAINING CAPABILITIES

$$\text{objective}(\phi) = E_{(x,y)\sim D_{\pi_\phi^{\mathrm{RL}}}}\left[r_\theta(x,y) - \beta \log\left(\pi_\phi^{\mathrm{RL}}(y \mid x)/\pi^{\mathrm{SFT}}(y \mid x)\right)\right] + \gamma E_{x\sim D_{\text{pretrain}}}\left[\log(\pi_\phi^{\mathrm{RL}}(x))\right]$$

- The figure shows the **PPO-ptx variant**: add a pretraining log-likelihood term to the RLHF objective.
- Its coefficient γ in this historical figure is a mixture weight, **not** the RL discount factor.
- The reference-model KL limits drift; the pretraining mixture helps preserve capabilities learned before SFT.
- These are choices of training objective, separate from PPO's update mechanism.

PPO: UPDATING THE POLICY IN SMALL STEPS

- Sample responses with a snapshot π_{old}; estimate advantages $\widehat{A}_t$ with a critic.
- Compare the new policy to the policy that collected the data:

$$\rho_t(\phi) = \frac{\pi_\phi(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}.$$

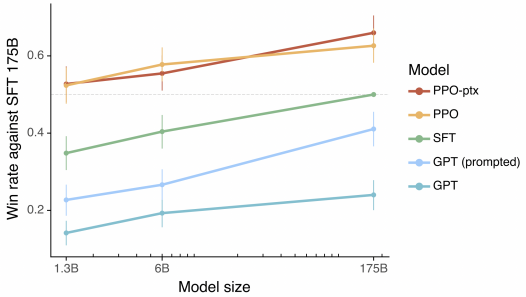
- PPO-Clip maximises a conservative surrogate:

$$\mathbb{E}_t\left[\min\left(\rho_t\widehat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\widehat{A}_t\right)\right].$$

- Clipping removes incentives for overly large probability changes in the improving direction; it is not a hard bound on policy change.
- **Two comparisons:** PPO compares against the recent sampling policy; RLHF's KL penalty compares against the frozen SFT reference.

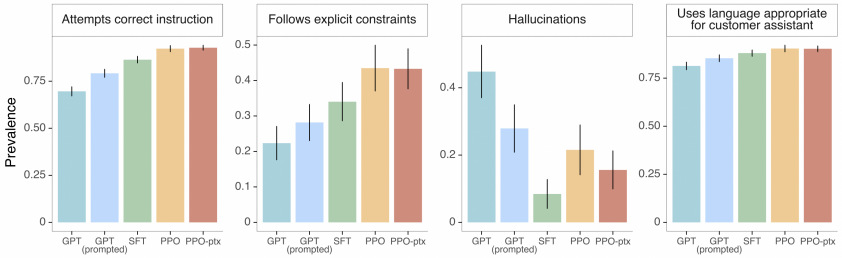
Evaluation

MAIN EVALUATION RESULT



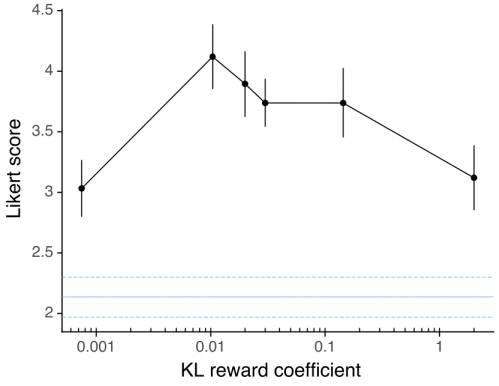
- PPO-ptx: tries to preserve behavior on pretraining data (the γ term) $\rightarrow$ less regression on public NLP datasets.
- SFT and PPO are not that different in performance?
  - PPO win rate is "only" about .6.
  - So PPO wins in 6 cases, SFT in 4.

IMPROVEMENT ON FOUR DIMENSIONS



- PPO models better than GPT throughout
- SFT better than PPO on hallucinations. **Question:** Why?

IMPORTANT HYPERPARAMETER: KL REWARD COEFFICIENT BETA



Limitations

ATTACK SUFFIXES

Universal and Transferable Adversarial Attacks on Aligned Language Models

Andy Zou[1], Zifan Wang[2], J. Zico Kolter[1,3], Matt Fredrikson[1]
[1]Carnegie Mellon University, [2]Center for AI Safety, [3]Bosch Center for AI
andyzou@cmu.edu, zifan@safe.ai, zkolter@cs.cmu.edu, mfredrik@cs.cmu.edu

- Can "aligned" LLMs be made to produce unsafe content?
- Attack suffix: is appended to prompt to circumvent guardrails
- Historically, some attack suffixes transferred across models.
- Attack suffixes show brittleness of RLHF / posttraining in general

FACTUALITY: PREFERENCE IS NOT TRUTH

- A fluent, persuasive response can contain false or unsupported factual claims.
- Human preferences and the learned reward model are imperfect proxies for factual correctness.
- Optimising those preferences does not guarantee truthfulness; evaluate factuality separately.
- Creative fiction is not a factual claim. Distinguish factual errors from failures to follow the input faithfully.

Connection to RL

An agent optimises the reward signal it receives; that signal may miss the property we actually care about.

WHOSE PREFERENCES ARE BEING LEARNED?

- Preference labels reflect the annotators, their instructions and the prompts sampled for training.
- A policy can match the training labelers while failing to represent other users or contexts.
- Helpfulness, truthfulness and harmlessness can conflict; the labeling rubric sets tradeoffs.
- Report whose judgments are represented and evaluate on held-out people, tasks and contexts.

Question: How would you evaluate a policy for users whose preferences differ from the training annotators?

Reward Hacking and Verifiable Rewards

REWARD HACKING

- Reward hacking: increasing the proxy reward without improving the intended outcome.
- In RLHF, polished or confident responses may score highly despite factual errors.
- The learned reward model can become unreliable on outputs produced by an increasingly optimised policy.
- A reference-policy KL penalty limits drift and can mitigate over-optimisation; it does not guarantee correctness or safety.
- Evaluate the real goal independently of the reward used for training.

BACK TO SESSION 8: VERIFIABLE REWARDS AND GRPO

- Session 8 introduced reasoning trajectories rewarded for a correct answer or executable result.
- **Reward source:** a verifier can replace a human-preference reward model when outcomes can be checked.
- **Optimiser:** GRPO (Group Relative Policy Optimization) compares sampled responses to the same prompt to estimate an advantage, without a separate value critic.
- GRPO can use learned or rule-based rewards: the optimiser and reward source are separate choices.
- The same loop remains: sample responses, score them, and update the policy. A correct answer alone does not verify every reasoning step.

▸ DeepSeekMath: GRPO

LICENSE & ATTRIBUTION

- **RL foundations:** 75 frames adapted from David Silver, *UCL Course on Reinforcement Learning* (2015), Lectures 1, 2, 5, 6 and 7. `https://davidstarsilver.wordpress.com/teaching/`. **CC BY-NC 4.0**; these adaptations are shared under CC BY-NC 4.0. Third-party figures are excluded from the source license; DQN figures credited on their frames.
- **RLHF application:** 20 frames adapted from DL4NLP, SLDS, LMU Munich, chapter 10, **CC BY 4.0**; selection, corrections and rewrites documented in `order.txt`.
- **Course additions:** learning goals and three bridges (MC/TD targets, LLM mapping, PPO update). PPO clarification: Schulman et al. (2017), PPO; InstructGPT: Ouyang et al. (2022), InstructGPT; GRPO: Shao et al. (2024), DeepSeekMath.